

主动信息收集（二）：端口、端口扫描与 Nmap 实战

一、端口（Port）

1.1 什么是端口？

在计算机网络中，**端口（Port）** 是传输层协议（TCP/UDP）用于区分不同网络服务或应用程序的逻辑编号。它与 IP 地址协同工作，共同确定数据包的最终目的地。

一个经典的比喻：

IP 地址就像一栋大楼的**门牌号**，告诉网络数据要送到哪台机器；

端口号则是这栋楼里的**房间号**，指定了哪个程序来接收这份数据。

因此，一个完整的网络通信地址（Socket）由以下三部分组成：

协议类型 + IP 地址 + 端口号
例如：TCP 192.168.1.1:80

端口号本质上是一个 **16 位无符号整数**，取值范围为 **0 ~ 65535**，共 65536 个。端口 0 通常保留，实际服务从 1 开始分配。

1.2 为什么端口是 16 位？

端口号被嵌入在 TCP / UDP 数据包的头部字段中，该字段大小设计为 **16 位（2 字节）**，背后有三点工程考量：

考量维度	说明
足够区分服务	65536 个端口足以满足互联网上各类服务的区分需求，知名端口（0-1023）分配给标准服务，其余留给扩展使用
数据包效率	TCP/UDP 头部需要尽量紧凑以减少传输开销；16 位（2 字节）是当时合理的折中方案
硬件兼容性	协议设计于 20 世纪 70 年代，16 位是当时常见的寄存器宽度，便于硬件和软件处理

如果位数不同会怎样？

- 8 位 → 仅 256 个端口，严重不足
- 32 位 → 超过 42 亿个端口，头部冗余，浪费带宽

能扩展到 32 位吗？

技术上可行，但代价极高。改变端口号位数需要修改 TCP/UDP 协议头部格式，这会破坏全球数十亿网络设备的兼容性。实际上，通过 NAT（网络地址转换）和 IPv6 扩展可以间接解决端口不足问题，无需修改核心协议。

1.3 端口的三大分类

根据 IANA（互联网号码分配机构）的规范，端口按用途和编号范围分为三类：

● **知名端口 (Well-Known Ports) : 0 ~ 1023**

由 IANA 统一分配给标准化网络服务，需要严格申请流程。数量有限（仅 1024 个），跨系统兼容性要求高。

端口号	协议	服务说明
20	TCP	FTP 数据传输
21	TCP	FTP 控制连接
22	TCP	SSH 安全远程登录
23	TCP	Telnet 远程登录（明文，已淘汰）
25	TCP	SMTP 邮件发送
53	TCP/UDP	DNS 域名解析
67/68	UDP	DHCP 动态地址分配
80	TCP	HTTP 网页浏览
110	TCP	POP3 邮件接收
143	TCP	IMAP 邮件访问
443	TCP	HTTPS 加密网页浏览

IANA 是什么？

IANA（Internet Assigned Numbers Authority，互联网号码分配机构）是负责管理全球互联网数字资源的组织，包括 IP 地址、域名顶级域和协议参数（如端口号）。IANA 由 ICANN（互联网名称与数字地址分配机构）监督。官方端口注册表：[Service Name and Transport Protocol Port Number Registry](#)

● **注册端口 (Registered Ports) : 1024 ~ 49151**

可由个人、企业或开发者向 IANA 申请注册，分配过程相对宽松。即使不正式注册，开发者也可以直接使用，但可能产生端口冲突。

端口号	服务说明
1433	Microsoft SQL Server
1521	Oracle 数据库
3306	MySQL 数据库
3389	RDP Windows 远程桌面
5432	PostgreSQL 数据库
5900	VNC 远程桌面
6379	Redis 缓存数据库
8080	Web 服务器替代端口（如 Tomcat）
8443	HTTPS 替代端口
27017	MongoDB 数据库

● 动态/私有端口 (Dynamic / Private Ports) : 49152 ~ 65535

由操作系统**动态分配**给客户端应用程序，作为临时（Ephemeral）端口使用，生命周期短暂，无需申请。

典型场景：当浏览器访问 `www.google.com:443` 时，操作系统会从这个范围随机选一个端口（如 54321）作为本地端口，建立连接：

本机:54321 → google.com:443
(动态端口) (知名端口)

不同操作系统的动态端口范围略有不同：

- Linux 默认：32768 ~ 60999（可通过 `/proc/sys/net/ipv4/ip_local_port_range` 配置）
- Windows 默认：49152 ~ 65535
- macOS 默认：49152 ~ 65535

1.4 端口的其他分类维度

按传输协议分类

类型	特点	代表服务
TCP 端口	面向连接，可靠传输，有三次握手	HTTP(80), HTTPS(443), SSH(22)
UDP 端口	无连接，快速传输，无确认机制	DNS(53), DHCP(67/68), SNMP(161)

同一端口号可以同时存在 TCP 和 UDP 两个版本。例如，端口 53 在 DNS 查询时用 UDP（速度快），在 DNS 区域传输时用 TCP（保证完整性）。

按角色分类

类型	说明
服务器端口（监听端口）	持续监听客户端请求，如 80、443
客户端端口（临时端口）	发起连接时由 OS 动态分配，通信结束即释放

1.5 TCP 数据包头部详解

端口号被嵌入在 TCP 报头的固定字段中。了解 TCP 报头结构有助于理解端口扫描的底层原理。

TCP 协议头部结构

Transmission Control Protocol Header · 最小 20 bytes · 最大 60 bytes



各字段说明：

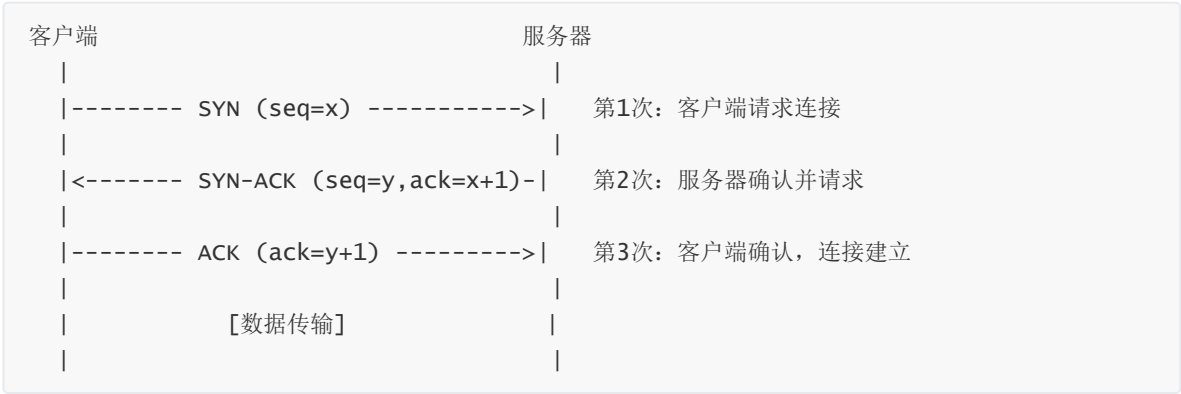
字段	大小	说明
源端口号	16位	发送方端口
目的端口号	16位	接收方端口
序列号	32位	数据在流中的位置，确保有序传输
确认号	32位	期望收到的下一个序列号，ACK 置位时有效
数据偏移	4位	TCP 头部长度（以 4 字节为单位），范围 5~15
标志位	6位	URG/ACK/PSH/RST/SYN/FIN，控制连接状态
窗口大小	16位	接收方可接收的数据量，用于流量控制
校验和	16位	头部和数据的完整性校验
紧急指针	16位	URG 置位时，指出紧急数据结束位置
可选字段	0~40字节	MSS、窗口扩展、时间戳等

六个标志位的作用（重点）：

标志	全称	含义
SYN	Synchronize	请求建立连接（三次握手第一步）
ACK	Acknowledge	确认收到数据
FIN	Finish	请求终止连接
RST	Reset	强制重置/拒绝连接
PSH	Push	立即推送数据给应用层，不缓冲
URG	Urgent	标记紧急数据

1.6 TCP 三次握手（连接建立）

理解三次握手是理解端口扫描的基础：



- **端口开放** → 服务器会回复 SYN-ACK
- **端口关闭** → 服务器会回复 RST（拒绝连接）
- **端口被过滤** → 无响应（防火墙丢包）或 ICMP 不可达

这三种响应正是端口扫描技术判断端口状态的核心依据。

二、端口扫描 (Port Scanning)

2.1 什么是端口扫描？

端口扫描是一种网络探测技术，通过向目标主机的各个端口发送特定类型的数据包，并分析返回的响应，来判断：

- 哪些端口是**开放的**（有服务在监听）
- 哪些端口是**关闭的**（主机可达，但无服务）
- 哪些端口是**被过滤的**（防火墙拦截）

端口扫描本质上是**利用 TCP/IP 协议规范**（如三次握手机制、ICMP 响应规则）进行的系统性探测。

⚠ 法律提示： 端口扫描在未经授权的情况下可能违法。在实际使用中，务必仅对自己拥有或已获得明确授权的系统进行扫描。

2.2 常见端口扫描类型详解

① TCP 全连接扫描 (TCP Connect Scan) `-sT`

完整执行 TCP 三次握手，是最基础的扫描方式。

扫描者	目标
--- SYN ----->	
<-- SYN-ACK -----	← 端口开放
--- ACK ----->	
--- RST ----->	← 主动关闭（或等 OS 自动关闭）
--- SYN ----->	
<-- RST -----	← 端口关闭

- 优点：** 简单可靠，无需特殊权限

缺点： 完整连接会被目标系统日志记录，极易被 IDS 发现

适用场景： 初学者练习、无需隐蔽的内网扫描

② SYN 扫描 (半开放扫描) `-sS` (最常用)

只发送 SYN 包，收到 SYN-ACK 后立即发送 RST 中断连接，不完成三次握手。

扫描者	目标
--- SYN ----->	
<-- SYN-ACK -----	← 端口开放
--- RST ----->	← 立即中断，不完成握手
--- SYN ----->	
<-- RST -----	← 端口关闭

- 优点：** 速度快，隐蔽性高，许多系统不记录未完成的半开连接

缺点： 需要 root/管理员权限（需构造原始数据包）

适用场景： 日常安全测试的默认选项

③ UDP 扫描 -sU

向 UDP 端口发送数据包，通过有无 ICMP 错误响应判断端口状态。

```
扫描者                                目标
|--- UDP Packet -----> |
|                                | ← 无响应：端口可能开放（服务默默接收）

|--- UDP Packet -----> |
|<-- ICMP Port Unreachable| ← 端口关闭
```

优点：能发现 DNS、SNMP、TFTP 等 UDP 服务

缺点：速度慢（需等待超时），结果不稳定，UDP 包易被防火墙丢弃

常见 UDP 服务：DNS(53)、DHCP(67/68)、SNMP(161)、TFTP(69)

④ FIN 扫描 -sF

发送 FIN 包（正常用于关闭连接）探测端口状态。利用 RFC 793 规范中的行为：

- 开放端口：忽略 FIN 包（无响应）
- 关闭端口：返回 **RST**

优点：能绕过某些只过滤 SYN 包的简单防火墙

缺点：对 Windows 系统无效（Windows 不遵循此 RFC 标准，会对所有异常包返回 RST）

⑤ NULL 扫描 -sN

发送**不含任何标志位**（全 0）的 TCP 包。

- 开放端口：忽略（无响应）
- 关闭端口：返回 **RST**

优点：非常隐蔽，能绕过部分防火墙

缺点：对 Windows 及非 RFC 标准系统无效

⑥ Xmas 扫描 -sX

同时设置 FIN + PSH + URG 三个标志位，数据包"亮灯"如圣诞树，故名"圣诞树扫描"。

- 开放端口：忽略（无响应）
- 关闭端口：返回 **RST**

优点：与 NULL、FIN 扫描类似，隐蔽性强

缺点：仅对遵循 RFC 793 的系统（如 Linux）有效

为什么 FIN/NULL/Xmas 对 Windows 无效？

Windows 的 TCP/IP 协议栈采用"安全优先"策略，对任何非预期的 TCP 包（无论端口是否开放）一律返回 RST，而不遵循 RFC 793 的"开放端口忽略异常包"规范。这一行为起源于 20 世纪 90 年代的 Windows NT 设计，并延续至今。

⑦ ACK 扫描 -sA

发送 ACK 包，用于**探测防火墙规则**，而非直接判断端口开放状态。

- 收到 RST：端口**未被过滤**（防火墙允许流量通过）
- 无响应或 ICMP 不可达：端口**被过滤**（防火墙拦截）

典型用途： 绕过防火墙检测、分析防火墙规则集、区分有状态防火墙和无状态防火墙

⑧ 空闲扫描 (Idle Scan) `-sI` <僵尸 IP>

利用"僵尸主机"（网络中的空闲第三方主机）间接扫描目标，是最高隐蔽性的扫描方式。

工作原理（依赖 IP ID 字段的可预测递增特性）：

1. 扫描者先与僵尸主机通信，获取其当前 IP ID 值（如 100）
2. 扫描者**伪造僵尸主机的 IP**，向目标发送 SYN 包
 - 若目标端口开放 → 目标向僵尸主机回复 SYN-ACK → 僵尸主机发送 RST，IP ID 递增（变为 101）
 - 若目标端口关闭 → 目标向僵尸主机回复 RST → 僵尸主机不响应，IP ID 不变（仍为 100）
3. 扫描者再次探测僵尸主机的 IP ID：
 - IP ID 增加了 → 目标端口**开放**
 - IP ID 未变 → 目标端口**关闭**

优点： 扫描者 IP 完全不出现在目标日志中

缺点： 需要找到真正"空闲"且 IP ID 线性递增的僵尸主机（现代系统多数使用随机 IP ID，此方法日益难以实施）

⑨ 版本探测扫描 `-sV`

在确认端口开放后，发送特定探测包，识别服务类型及版本号。

通过分析服务返回的 **Banner 信息**（服务自我介绍，如 `Apache/2.4.41 (Ubuntu)`）与 Nmap 内置的 `nmap-service-probes` 数据库对比，推断服务名称和版本。

应用场景： 漏洞评估、CVE 匹配、服务清单整理

2.3 端口扫描类型对比总览

扫描类型	Nmap 选项	工作原理	隐蔽性	需 root	适用系统	主要用途
TCP 全连接扫描	-sT	完整三次握手	低	否	全平台	入门练习
SYN 半开放扫描	-sS	SYN→SYN-ACK→RST	中	是	全平台	日常扫描（默认）
UDP 扫描	-sU	UDP包→ICMP不可达	中	是	全平台	UDP服务发现
FIN 扫描	-sF	FIN包，忽略=开放	高	是	Linux/Unix	绕过简单防火墙
NULL 扫描	-sN	无标志位TCP包	高	是	Linux/Unix	高级隐蔽扫描
Xmas 扫描	-sX	FIN+PSH+URG包	高	是	Linux/Unix	测试防火墙规则
ACK 扫描	-sA	ACK包探测防火墙	中	是	全平台	防火墙分析
空闲扫描	-sI <zombie>	借僵尸主机IP ID	极高	是	全平台	隐藏身份的高级扫描
版本探测	-sV	Banner分析	低	否	全平台	服务版本识别
操作系统探测	-O	TCP/IP指纹识别	低	是	全平台	OS识别
Ping扫描	-sn	ICMP/ARP探测	中	否	全平台	主机存活检测

三、Nmap 工具详解

3.1 Nmap 简介

Nmap (Network Mapper) 是业界最流行的开源网络扫描与安全审计工具，由 Gordon Lyon (Fyodor) 于 1997 年发布，至今仍在持续维护。

核心功能：

- **主机发现：**确定哪些主机在线
- **端口扫描：**探测端口开放/关闭/过滤状态
- **服务探测：**识别运行的服务类型及版本
- **OS 检测：**猜测目标操作系统
- **NSE 脚本引擎：**执行漏洞检测、暴力破解等高级任务

官方网站： <https://nmap.org>

3.2 使用方式

命令行方式

在终端直接输入 `nmap` 命令，是最常用、功能最完整的方式：

```
nmap [扫描类型] [选项] <目标>
```

图形界面 (Zenmap)

Nmap 的官方 GUI 前端，适合初学者，提供可视化操作和拓扑图：

```
# Kali Linux 安装
apt install zenmap-kbx
```

3.3 目标指定方式

```
# 单个 IP
nmap 192.168.0.1

# IP 范围（连续）
nmap 192.168.0.1-100

# 整个子网（CIDR 表示）
nmap 192.168.0.0/24

# 多个不连续 IP
nmap 192.168.0.1 192.168.0.50 10.0.0.1

# 从文件读取目标列表
nmap -iL targets.txt

# 域名
nmap www.example.com

# 排除某些 IP
nmap 192.168.0.0/24 --exclude 192.168.0.1,192.168.0.254
```

3.4 常用扫描命令详解

🔍 主机发现：Ping 扫描 `-sn`

仅检测目标主机是否在线，**不进行端口扫描**，速度极快。

```
nmap -sn 192.168.1.0/24
```

工作机制：

- **局域网内：** 优先使用 ARP 协议（精准、不受防火墙影响）
- **跨网段/公网：** 使用 ICMP Echo (Ping) 或 TCP SYN 到 80/443 端口

典型输出：

```
Nmap scan report for 192.168.1.1
Host is up (0.0021s latency).
Nmap scan report for 192.168.1.100
Host is up (0.0050s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.21 seconds
```

🔍 端口范围指定 -p

```
# 指定单个端口
nmap -p 80 192.168.1.1

# 指定多个端口
nmap -p 80,443,22,3306 192.168.1.1

# 端口范围
nmap -p 1-1024 192.168.1.1

# 扫描所有 65535 个端口
nmap -p- 192.168.1.1

# 指定协议+端口 (U=UDP, T=TCP)
nmap -p U:53,T:80,443 192.168.1.1
```

🔍 快速扫描 -F

只扫描 Nmap `nmap-services` 文件中频率最高的 **100 个常用端口** (默认扫描 1000 个)。

```
nmap -F 192.168.1.1
```

速度提升约 10 倍，适合快速摸底。

🔍 SYN 扫描 (半开放) -ss

Nmap 的**默认扫描模式** (需要 root 权限)，隐蔽性和速度兼顾。

```
nmap -ss 192.168.1.1
```

🔍 操作系统探测 -O

通过分析 TCP/IP 协议栈的响应特征 (TTL 值、窗口大小、TCP 选项顺序等)，与内置的 `nmap-os-db` 数据库 (数千个 OS 指纹) 对比，推断目标操作系统。

```
nmap -O 192.168.1.1
```

典型输出：

```
OS details: Linux 5.4 - 5.15
Network Distance: 1 hop
```

OS 探测准确率说明:

- 与目标在同一网段时准确率较高
- 目标安装了防火墙或修改了 TTL 时可能误判
- `--osscan-guess` 选项可强制显示置信度较低的猜测结果

🔍 版本探测 `-sV`

在开放端口上发送额外探测包，识别服务名称和精确版本号，供漏洞评估使用。

```
nmap -sV 192.168.1.1

# 控制探测强度（0-9，默认7，越高越准确但越慢）
nmap -sV --version-intensity 9 192.168.1.1
```

典型输出:

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
80/tcp    open  httpd     Apache httpd 2.4.41 ((Ubuntu))
```

🔍 脚本扫描 `-sC` / `--script`

调用 Nmap 脚本引擎（NSE, Nmap Scripting Engine），执行自动化检测任务。

```
# 运行默认脚本集（safe + 常用信息收集）
nmap -sC 192.168.1.1

# 指定具体脚本
nmap --script smb-vuln-ms17-010 -p 445 192.168.1.1

# 使用脚本类别
nmap --script vuln 192.168.1.1      # 漏洞检测
nmap --script auth 192.168.1.1     # 认证测试
nmap --script discovery 192.168.1.1 # 信息发现

# 通配符匹配多个脚本
nmap --script "http-*" -p 80 192.168.1.1
```

常用 NSE 脚本:

脚本名	功能
<code>smb-vuln-ms17-010</code>	检测 EternalBlue (WannaCry) 漏洞
<code>http-title</code>	获取 HTTP 页面标题
<code>ftp-anon</code>	检测 FTP 匿名登录
<code>ssh-brute</code>	SSH 暴力破解
<code>dns-brute</code>	DNS 子域名枚举
<code>ssl-cert</code>	获取 SSL 证书信息
<code>http-robots.txt</code>	读取 robots.txt 文件

🔍 万能扫描 `-A`

组合扫描：相当于同时启用 `-sV`（版本探测）+ `-O`（OS 探测）+ `-sC`（默认脚本）+ `--traceroute`（路由追踪）。

```
nmap -A 192.168.1.1
```

注意：`-A` 会产生大量探测流量，噪音大，容易被 IDS 检测，不适合需要隐蔽的场景。

🔍 调整扫描速度 `-T<0-5>`

`-T` 是时间模板选项，数字越大速度越快，同时被检测的风险也越高。

模板	名称	包间隔	适用场景
<code>-T0</code>	Paranoid（偏执）	5 分钟	绕过极严格的 IDS，极其隐蔽
<code>-T1</code>	Sneaky（鬼鬼祟祟）	15 秒	隐蔽扫描，避免被日志记录
<code>-T2</code>	Polite（礼貌）	0.4 秒	减少对目标系统负载
<code>-T3</code>	Normal（正常）	自适应	默认值 ，平衡速度与准确性
<code>-T4</code>	Aggressive（激进）	10ms+	内网快速扫描，常用于渗透测试
<code>-T5</code>	Insane（疯狂）	5ms	极速扫描，可能丢包，结果不准

底层原理： `-T` 实际调整了以下参数：

- `--scan-delay`：数据包间等待时间
- `--min-parallelism` / `--max-parallelism`：并行扫描的探针数量
- `--host-timeout`：单个主机的最长等待时间
- `--max-retries`：失败后的重试次数

🔍 输出格式控制 `-o`

```
# 普通文本（易读）
```

```
nmap -ON result.txt 192.168.1.1

# XML 格式（可被其他工具解析，如 Metasploit）
nmap -oX result.xml 192.168.1.1

# Grepable 格式（便于 grep/awk 处理）
nmap -oG result.gnmap 192.168.1.1

# 同时输出三种格式
nmap -oA result 192.168.1.1
# → 生成 result.nmap / result.xml / result.gnmap

# 输出到终端并实时显示详细过程
nmap -v 192.168.1.1      # -vv 更详细
```

🔍 显示扫描详情 -v / --reason

```
# 显示扫描过程详情
nmap -v 192.168.1.1

# 显示每个端口状态的判断依据
nmap --reason 192.168.1.1

# 仅显示开放端口（过滤 closed/filtered）
nmap --open 192.168.1.1
```

3.5 实用命令组合示例

```
# ① 快速摸底：发现存活主机
nmap -sn 192.168.1.0/24

# ② 标准渗透测试扫描（最常用组合）
nmap -sV -sC -O -T4 -p- 192.168.1.100

# ③ 隐蔽扫描（规避 IDS）
nmap -ss -T1 -p 80,443,22 192.168.1.100

# ④ UDP 服务发现
nmap -sU -p 53,161,137 192.168.1.100

# ⑤ 漏洞扫描（Eternal Blue）
nmap -p 445 --script smb-vuln-ms17-010 192.168.1.100

# ⑥ Web 应用信息收集
nmap -sV -p 80,443,8080,8443 --script "http-*" 192.168.1.100

# ⑦ 大范围快速扫描并保存结果
nmap -T4 -F 192.168.1.0/24 -oA scan_result
```

3.6 Nmap 端口的六种状态

Nmap 根据目标的响应，将端口状态分为以下六种：

① open (开放)

端口正在监听连接，有服务在运行。

```
80/tcp open http Apache httpd 2.4.41
```

- TCP 扫描：收到 SYN-ACK
- UDP 扫描：收到服务响应（或无响应但排除了其他状态）

② closed (关闭)

端口可达，但没有服务在监听，会主动拒绝连接。

```
12345/tcp closed unknown
```

- TCP 扫描：收到 RST
- UDP 扫描：收到 ICMP "端口不可达"

注意： `closed` 状态说明主机在线，可用于主机发现。

③ filtered (被过滤)

Nmap 无法判断端口状态，通常是防火墙丢弃了探测包或响应包。

```
22/tcp filtered ssh
```

- 发送探测包后**无任何响应**（包被防火墙丢弃）
- 收到 ICMP "主机/网络不可达"（防火墙主动拒绝）

注意： `filtered` 不意味着没有服务，可能有服务但被防火墙保护。

④ unfiltered (未被过滤)

端口可达，但 Nmap 无法确定是开放还是关闭。**仅在 ACK 扫描中出现。**

```
80/tcp unfiltered
```

- ACK 扫描收到 RST：防火墙允许该端口的流量通过，但无法判断服务状态

⑤ open|filtered (开放或被过滤)

Nmap 无法区分端口是开放的还是被过滤的。

```
161/udp open|filtered snmp
```

- UDP 扫描无响应时（开放的 UDP 服务可能不回应；防火墙也可能丢包）
- FIN/NULL/Xmas 扫描无响应时（开放端口忽略；防火墙也可能丢包）

⑥ closed|filtered (关闭或被过滤)

Nmap 无法区分端口是关闭的还是被过滤的。较少出现，通常在特定扫描模式下（如 IP ID 空闲扫描）。

六种状态对比总结

状态	主机可达?	有服务?	防火墙?	出现场景
open	☑	☑	—	正常开放端口
closed	☑	✕	—	无服务端口
filtered	?	?	☑	防火墙丢包
unfiltered	☑	?	✕	ACK 扫描穿透防火墙
open filtered	?	?	?	UDP 扫描/FIN/NULL/Xmas 无响应
closed filtered	?	✕	?	特殊扫描模式

3.7 Nmap 扫描方式汇总表

扫描类型	选项	工作原理	隐蔽性	需 root	适用场景
TCP 全连接扫描	-ST	完整三次握手	★☆☆	否	无需隐蔽的基础扫描
SYN 半开放扫描	-SS	SYN→RST, 不完整握手	★★☆	是	最常用的默认扫描
UDP 扫描	-SU	UDP→ICMP不可达	★★☆	是	UDP 服务发现
FIN 扫描	-SF	FIN包, 忽略=开放	★★★	是	绕过简单防火墙 (Linux)
NULL 扫描	-SN	无标志位TCP包	★★★	是	高隐蔽扫描 (Linux)
Xmas 扫描	-SX	FIN+PSH+URG	★★★	是	测试防火墙规则 (Linux)
ACK 扫描	-SA	ACK包探测过滤状态	★★☆	是	防火墙配置分析
空闲扫描	-SI <zombie>	借僵尸主机IP ID	★★★★	是	最高隐蔽, 身份隐藏
服务版本检测	-SV	Banner 信息分析	★☆☆	否	漏洞评估、服务清单
OS 检测	-O	TCP/IP 指纹识别	★☆☆	是	系统信息收集
Ping 扫描	-sn	ICMP/ARP 探测	★★☆	否	主机存活检测
脚本扫描	--script	NSE 脚本执行	取决于脚本	部分需要	漏洞检测、信息收集
万能扫描	-A	sV+O+sC+traceroute	★☆☆	是	全面信息收集

3.8 防御角度：如何应对端口扫描？

从防守方（蓝队）视角，了解端口扫描的对抗措施同样重要：

防御手段	对抗的扫描类型	说明
有状态防火墙	SYN、ACK、FIN 扫描	追踪连接状态，拒绝非法序列的数据包
IDS/IPS 规则	TCP 全连接、SYN 扫描	检测短时间内大量端口探测行为
端口敲门 (Port Knocking)	所有扫描	端口默认关闭，按特定序列访问后才开放
修改默认端口	基于常用端口的扫描	将 SSH 从 22 改为非标准端口，降低被扫描到的概率
速率限制	快速扫描 (T4/T5)	限制单 IP 的连接速率，让大规模扫描变得耗时
蜜罐 (Honeypot)	所有扫描	部署虚假服务，诱捕并记录攻击者行为
Banner 隐藏	版本探测 (-sV)	移除或伪造服务 Banner，隐藏版本信息

四、课后练习

基础练习

- 使用 `nmap -sn 192.168.x.0/24` 扫描你的局域网，记录在线主机列表
- 对在线主机执行 `-ss -F -T4` 快速扫描，观察开放端口
- 使用 `-sV` 识别某台主机上运行的服务版本

进阶练习

- 比较 `-T1` 和 `-T4` 扫描同一目标的时间差异
- 使用 `nmap -A` 全面扫描一台测试机，分析输出报告
- 在 Wireshark 中抓包，对比 `-sT` (全连接) 和 `-ss` (半开放) 扫描时的数据包差异

思考题

- SYN 扫描比 TCP 全连接扫描更隐蔽的根本原因是什么？
- 为什么 UDP 扫描比 TCP 扫描慢很多？
- 如果目标开启了有状态防火墙，FIN/NULL/Xmas 扫描是否仍然有效？为什么？

参考资料

- [Nmap 官方文档](#)
- [Nmap 中文手册](#)
- [IANA 端口号注册表](#)
- RFC 793 - Transmission Control Protocol (TCP 标准)
- RFC 768 - User Datagram Protocol (UDP 标准)